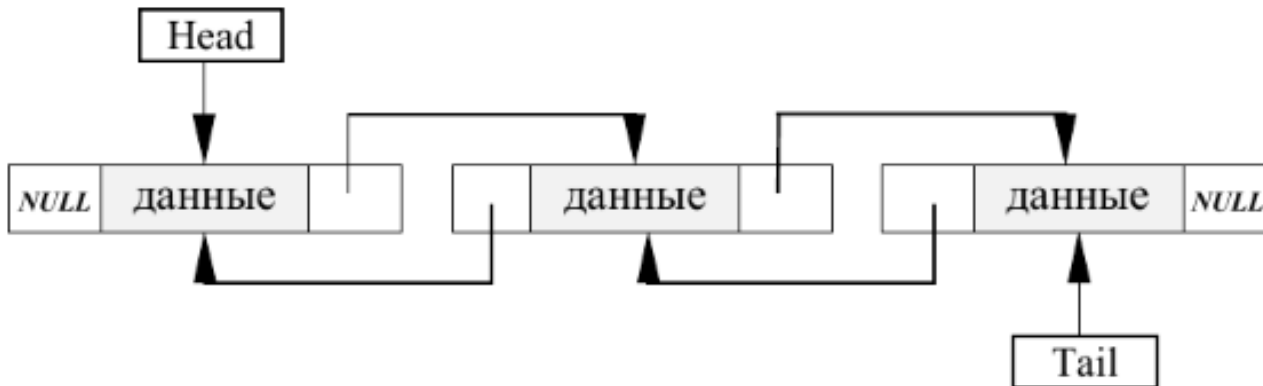


Лекция 9

Динамические структуры данных (часть 2)

3 Двусвязный список



Представление узла списка

```
struct Node
{
    int Data;
    Node *Next, *Prev;
};
...
Node *Head, *Tail;
```

```
class Node
{
    int Data;
    Node *Next, *Prev;
public:
    ...
};
```

Создание узла

```
struct Node // элемент списка
{
    int Data; // данные
    Node* Next; // указатель на следующую структуру
    Node* Prev; // указатель на предыдущую структуру
};

// создание узла
// d - данные узла
Node* CreateNode (int d)
{
    // указатель на новый узел
    Node * NewNode = new Node;
    // заполнение данными
    NewNode->Data = d;
    NewNode->Next = NULL;
    NewNode->Prev = NULL;
    return NewNode;
}
```

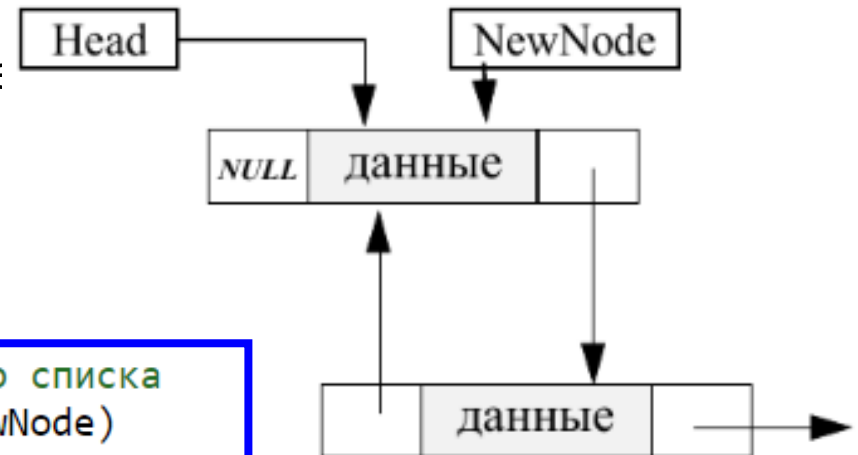
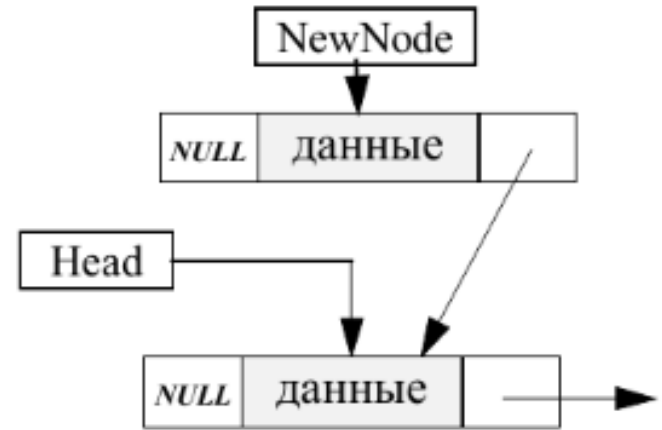
Проход по списку

Проход по двусвязному списку может выполняться в двух направлениях - от головы к хвосту (как для односвязного) или от хвоста к голове.

Добавление узла в начало спис

- 1) установить ссылку Next добавляемого узла NewNode на голову существующего списка и его ссылку Prev в NULL;
- 2) установить ссылку Prev бывшего первого узла (если он существовал) на NewNode;
- 3) установить голову списка на новый узел
- 4) если в списке не было ни одного элемента, хвост списка также устанавливается на новый узел.

```
// добавление узла в начало списка
void AddNodeStart(Node *NewNode)
{
    NewNode->Next = Head;
    NewNode->Prev = NULL;
    if ( Head ) Head->Prev = NewNode;
    Head = NewNode;
    if ( ! Tail ) Tail = Head;
}
```



```

1  #include <iostream>
2  using namespace std;
3
4  struct Node // элемент списка
5  {
6      int Data;    // данные
7      Node* Next;  // указатель на следующую структуру
8      Node* Prev;  // указатель на предыдущую структуру
9  };
10
11 // создание узла
12 // d - данные узла
13 Node* CreateNode (int d)
14 {
15     // указатель на новый узел
16     Node * NewNode = new Node;
17     // заполнение данными
18     NewNode->Data = d;
19     NewNode->Next = NULL;
20     NewNode->Prev = NULL;
21     return NewNode;
22 }
23
24 Node *Head, *Tail;
25

```

```

26 // печать списка
27 void SpisokPrint()
28 {
29     Node *p;
30     p = Head;
31     do { printf("%d ", p->Data);
32         p = p->Next;
33     } while (p != NULL);
34 }
35
36 // добавление узла в начало списка
37 void AddNodeStart(Node *NewNode)
38 {
39     NewNode->Next = Head;
40     NewNode->Prev = NULL;
41     if ( Head ) Head->Prev = NewNode;
42     Head = NewNode;
43     if ( ! Tail ) Tail = Head;
44 }
45
46 int main()
47 {
48     Node *NewNode1 = CreateNode(1);
49     AddNodeStart(NewNode1);
50     Node *NewNode2 = CreateNode(2);
51     AddNodeStart(NewNode2);
52     Node *NewNode3 = CreateNode(3);
53     AddNodeStart(NewNode3);
54     SpisokPrint();
55 }

```

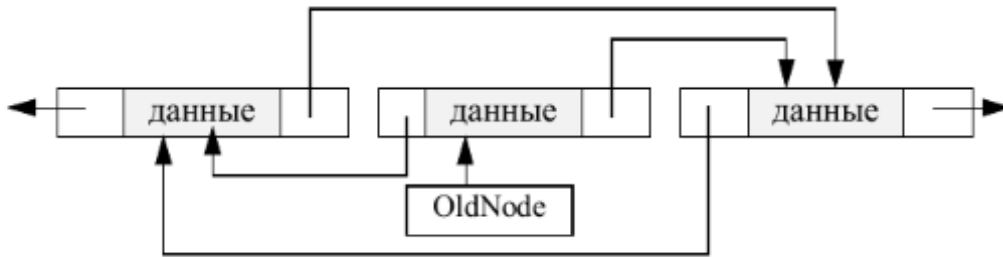
Добавление узла в конец списка?

Добавление узла после заданного?

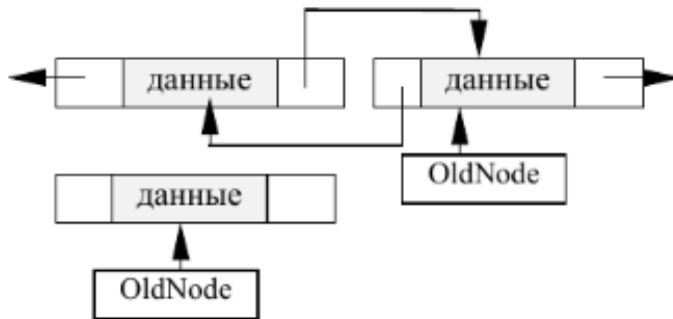
3 2 1

Удаление узла

1)



2)



```
// удаление узла
void Delete(Node *OldNode)
{
    if (Head == OldNode)
    {
        Head = OldNode->Next;
        if ( Head ) Head->Prev = NULL;
        else Tail = NULL;
    }
    else {
        OldNode->Prev->Next = OldNode->Next;
        if ( OldNode->Next )
            OldNode->Next->Prev = OldNode->Prev;
        else Tail = NULL;
    }
    delete OldNode;
}
```

```

46 // удаление узла
47 void Delete(Node *OldNode)
48 {
49     if (Head == OldNode)
50     {
51         Head = OldNode->Next;
52         if ( Head ) Head->Prev = NULL;
53         else Tail = NULL;
54     }
55     else {
56         OldNode->Prev->Next = OldNode->Next;
57         if ( OldNode->Next )
58             OldNode->Next->Prev = OldNode->Prev;
59         else Tail = NULL;
60     }
61     delete OldNode;
62 }
63
64 int main()
65 {
66     Node *NewNode1 = CreateNode(1);
67     AddNodeStart(NewNode1);
68     Node *NewNode2 = CreateNode(2);
69     AddNodeStart(NewNode2);
70     Node *NewNode3 = CreateNode(3);
71     AddNodeStart(NewNode3);
72     SpisokPrint();
73     cout << endl;
74     // удаление узла
75     Node *OldNode = NewNode2;
76     Delete(OldNode);
77     SpisokPrint();
78 }

```



```

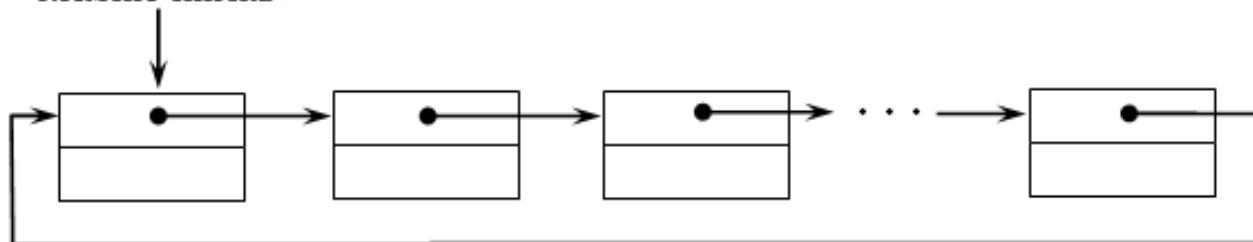
3 2 1
3 1

```

4 Циклические списки

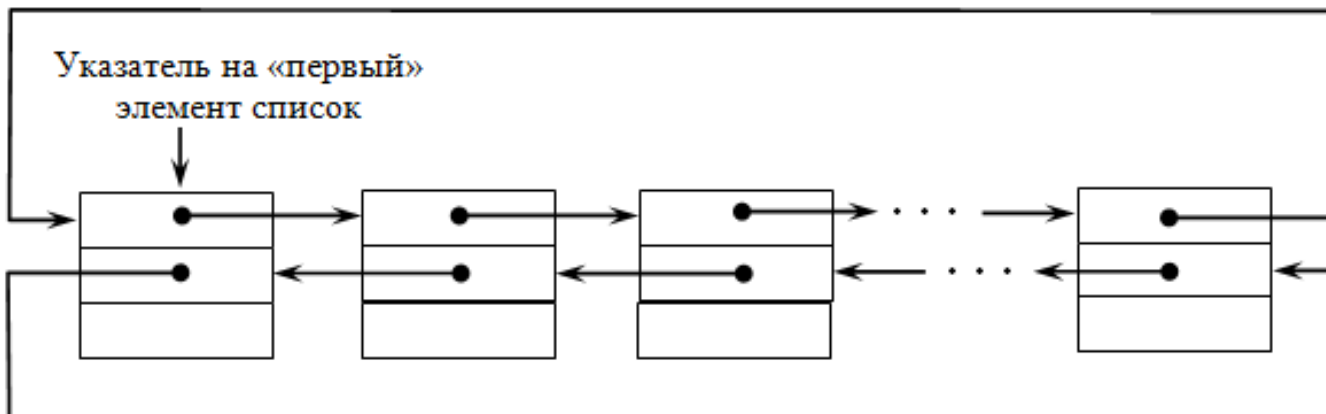
Циклический однонаправленный список

Указатель на «первый»
элемент списка



Циклический двунаправленный список

Указатель на «первый»
элемент список



Пример работы с циклическим однонаправленным списком

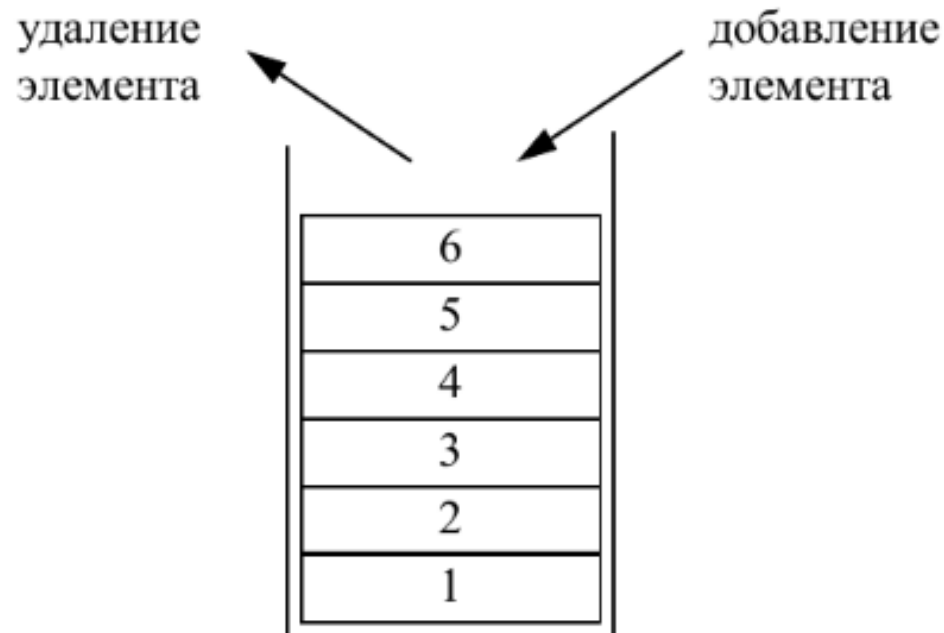
```
1  #include <iostream>
2  using namespace std;
3  //////////////////////////////////////////////////
4
5  struct Node // элемент списка
6  {
7      int Data; // данные
8      Node* Next; // указатель на следующую структуру
9  };
10
11  class List // список
12  {
13  private:
14      Node* Head;
15  public:
16      List() { Head = NULL; }
17      void AddNode(int d); // добавление элемента
18      void SpisokPrint(); // показ данных
19  };
20
21  // показ данных
22  void List::SpisokPrint()
23  {
24      Node* current = Head;
25      do {
26          // печатаем
27          cout << current->Data << endl;
28          // двигаемся к следующему элементу
29          current = current->Next;
30      }
31      // пока не дойдем до "головы"
32      while(current->Next != Head->Next);
33  }
```

```
34
35  // добавление элемента
36  void List::AddNode(int d)
37  {
38      Node* NewNode = new Node;
39      NewNode->Data = d;
40
41      // если в списке есть элемент(ы)
42      if(Head)
43      {
44          // добавление элемента после первого
45          NewNode->Next = Head->Next;
46          Head->Next = NewNode;
47      }
48      // в списке нет элементов
49      else {
50          // добавление элемента, который сам на себя ссылается
51          NewNode->Next = NewNode;
52          Head = NewNode;
53      }
54  }
55
56  int main()
57  {
58      List L; // создаем список
59      L.AddNode(1);
60      L.AddNode(2);
61      L.AddNode(3);
62      L.AddNode(4);
63
64      L.SpisokPrint(); // показываем список
65      return 0;
66  }
```

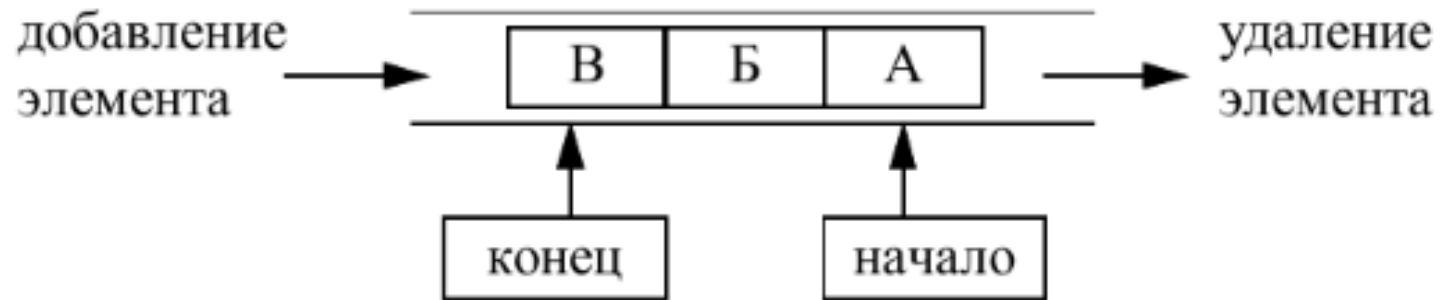
1
4
3
2

5 Понятия стека, очереди, дека и дерева

Стек - это упорядоченный набор элементов, в котором добавление новых и удаление существующих элементов допустимо только с одного конца, который называется вершиной стека.

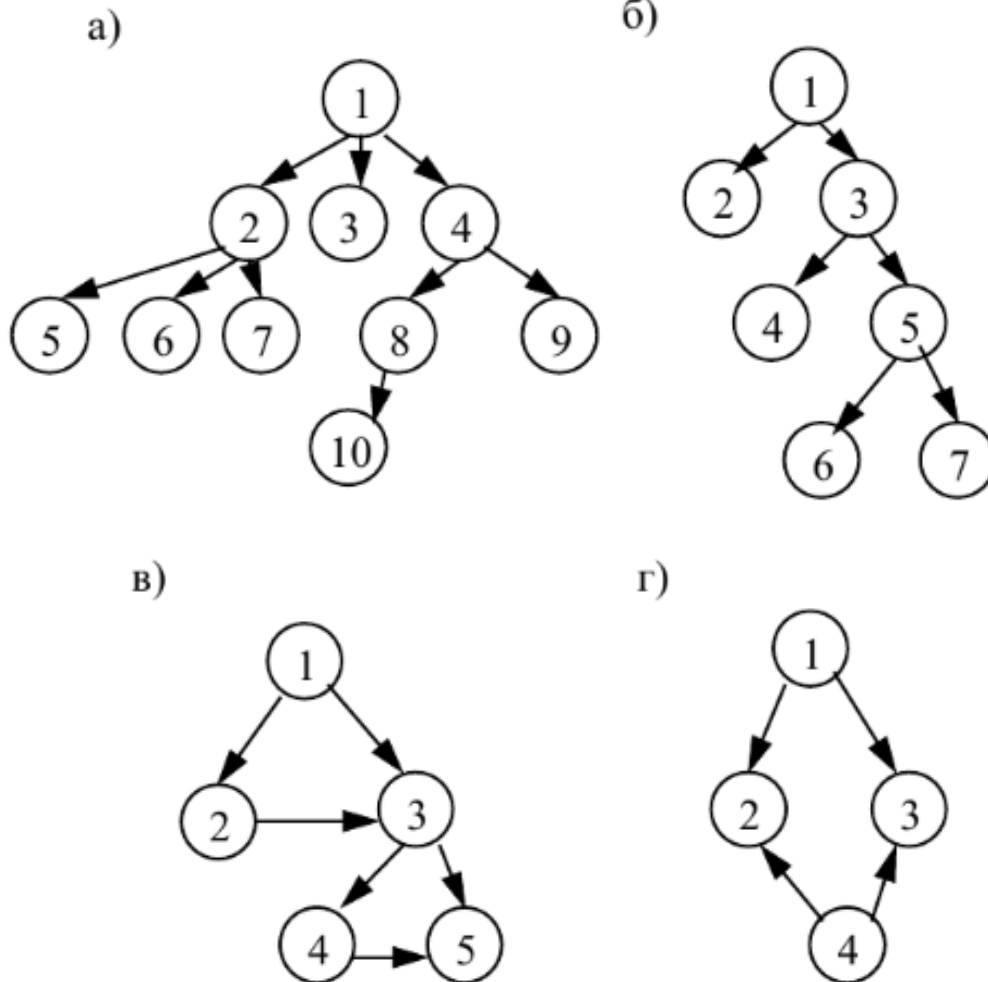


Очередь - это упорядоченный набор элементов, в котором добавление новых элементов допустимо с одного конца (он называется начало очереди), а удаление существующих элементов - только с другого конца, который называется концом очереди.



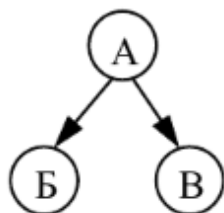
Дек - это упорядоченный набор элементов, в котором добавление новых и удаление существующих элементов допустимо с любого конца.

Дерево - это совокупность узлов (вершин) и соединяющих их направленных ребер (дуг), причем в каждый узел (за исключением одного - корня) ведет ровно одна дуга.

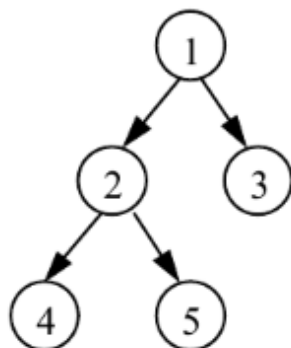


Двоичным деревом называется дерево, каждый узел которого имеет не более двух сыновей.

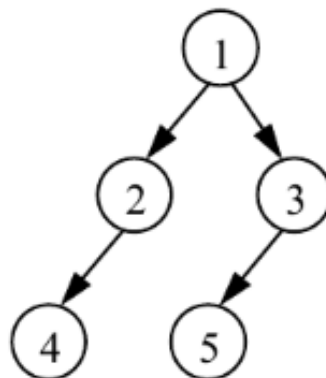
а)



б)



в)



г)

